

Distributed Rate Limiter

Stripe · AWS · Cloudflare · ప్రతి public API వెనక – "Redis lo ఒక counter పెట్టి, limit దాటిందా చూద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – `GET` తర్వాత `INCR` అనే ఒక్క ఆలోచన మీ పరిమితి 100 ని **1,058** చేస్తుంది. రెండోది – ఆ Redis పిలుపుని తప్పించుకోవడానికి local గా లెక్కపెడితే పరిమితి **20 రెట్లు** అవుతుంది. మూడోది – Redis పడిపోయినప్పుడు "fail open" అంటే మీ downstream కి **పరిమితి కంటే 500 రెట్లు** వెళ్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

LLD DEEP 03 (RATE LIMITER) చదివారా? – ఇది ఆ ALGORITHMS కథ కాదు

LLD Deep 03 lo నాలుగు algorithms ని లోతుగా చూశాం – fixed window యొక్క సరిహద్దు దాడి, sliding window log యొక్క memory ఖర్చు, sliding counter యొక్క ~1% లోపం, మరియు token bucket. అవన్నీ కొలిచి చూశాం, మరియు వాటిని ఇక్కడ మళ్ళీ చెప్పను.

HLD_Design_Problems §2 కూడా అదే ఐదు algorithms ని ఒక పట్టికలో ఇస్తుంది. ఆ పట్టిక సరైనదే.

ఈ doc మొదలయ్యేది సరిగ్గా అవి ఆగిన చోట, మరియు ఒక్క వాక్యంలో:

ఆ algorithms అన్న ఒక్క counter ఒక్క చోట ఉందని అనుకుంటాయి. Counter నిజానికి ఇంకో machine మీద ఉంది.

Token bucket `tokens` మరియు `lastRefill` చదివి, లెక్కించి, తిరిగి రాస్తుంది. ఒక్క process lo అది ఒక్క అడుగు. **50 servers మరియు ఒక Redis తో అది మూడు అడుగులు, మరియు వాటి మధ్య ప్రపంచం మారిపోతుంది** – §4 lo అది పరిమితిని **10 రెట్లు** చేస్తుంది.

అందుకే ఈ doc lo algorithm ఏదీ లేదు. **మూడు విరుపులూ counter ఎక్కడ ఉంది, ఎవరు దాన్ని ముట్టుకుంటున్నారో అనే దాని గురించే.**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏమి మారింది?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: రెండు అడుగుల మధ్య

3. Step – `GET` · check · `INCR`
4. మొదటి విరుపు – పరిమితి 100, అనుమతించినవి 1,058
5. Step – ఒకే ఆదేశం

Part 3 – రెండో విరుపు: ప్రతి request కీ ఒక round-trip

6. Step – local గా లెక్కపెట్టి అప్పుడప్పుడు sync చేయడం
7. రెండో విరుపు – పరిమితి 100, అనుమతించినవి 2,000
8. Step – quota ని పంచడం · మరియు దాని దాచిన అన్యాయం

Part 4 – మూడో విరుపు: Redis పడిపోతే

9. Step – fail open నా, fail closed నా?

10. మూడో విరుపు – రెండూ తప్పు

11. Step – చివరిసారి తెలిసిన వాటా

Part 5 – పూర్తి system

12. Step – ఒకే bucket, ఒకే budget

13. మొత్తం code · 48 లక్షల requests · **mutation testing**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటీతో చెప్పాల్సిన English script

16. Follow-ups – hot keys, బహుళ ప్రాంతాలు, tier-based limits

17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏమి మారింది?

"Rate limiter design చెయ్యి" అంటే అందరూ algorithms గురించి మాట్లాడతారు. కానీ ఈ problem **distributed** అని చెప్పగానే, algorithm ఎంపిక **సులభమైన భాగం** అవుతుంది.

ఎందుకంటే ప్రతి algorithm – token bucket అయినా, sliding counter అయినా – ఒకే ఆకారంలో ఉంటుంది:

చదువు → లెక్కించు → రాయి

ఒక్క process lo ఆ మూడూ ఒకే అడుగు. Counter మీ memory lo ఉంది, ఎవరూ దాన్ని మధ్యలో ముట్టుకోరు.

ఇప్పుడు 50 servers, మరియు counter ఒక Redis lo. ఆ మూడు అడుగులు **మూడు వేర్వేరు క్షణాలు** అయ్యాయి, మరియు వాటి మధ్య **వేలమంది** అదే counter ని చదువుతున్నారు.

ఒక్క process

చదువు → లెక్కించు → రాయు

ఒకే అడుగు · ఎవరూ మధ్యలో రారు

50 servers + Redis

చదువు ... లెక్కించు ... రాయు

GET

↔

నిర్ణయం

↔

INCR

ఆ రెండు ↔ గుర్తులే ఈ doc మొత్తం.

§4 – ఆ కీటికీలో వచ్చినవాళ్ళు అందరూ ఒకే పాత విలువ ని చూస్తారు.

§7 – ఆ విలువునే తప్పించుకుంటే, ప్రతి server కి సొంత సత్యం వస్తుంది.

§10 – ఆ Redis పడిపోతే, counter అనేదే లేకుండా పోతుంది.

మూడూ algorithm గురించి కాదు – దూరం గురించి.

అనుకుందాం: 1,00,000 QPS · 10 లక్షల active keys · 20 app servers · Redis same-AZ lo, RTT ~0.5 ms.

మొత్తం memory చిన్నదే – ఒక్క key కి ~50 bytes × 10 లక్షలు = **50 MB**. అది ఒక్క Redis node lo సునాయాసంగా పడుతుంది. కాబట్టి ఈ problem lo నిల్వ సమస్యే కాదు – సమస్య అంతా **ఏకకాలికత (concurrency)** మరియు **దూరం**.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
పరిమితి ఖచ్చితంగా ఉండాలా, ~1% లోపం ఘరవాలేదా?	ఇదే అతి ముఖ్యమైన ప్రశ్న. ఖచ్చితం అంటే ప్రతి request కీ ఒక Redis పీలుపు – §6
Redis పడిపోతే ఏమి చెయ్యాలి?	"Fail open" అనేది సహజమైన జవాబు. §10 lo అది 500x భారం అని కొలుస్తాం
Limiter ఎవరిని కాపాడుతోంది – మీ serverనా, downstream నా?	Downstream అయితే fail-open ఎప్పటికీ సరికాదు
Traffic అన్ని servers కి సమానంగా వస్తుందా?	రాకపోతే quota పంపకం 66% traffic ని అన్యాయంగా ఆపుతుంది – §8
Redis ఒకే AZ lo నా, వేరే region నా?	RTT 0.5 ms → 5 ms అయితే overshoot 89 → 958 – §4
ఒక్కే key కి traffic ఎంత?	ఒక key చాలా hot అయితే అది ఒకే Redis shard ని కొడుతుంది – §16

ఒక ప్రశ్న అడిగే విధానం మార్చండి

"Approximate సరిపోతుందా?" అని అడగడం సరిపోదు – అందరూ "అవును" అంటారు.

బదులుగా అడగండి: **"పరిమితి 100 అయితే, 110 వెళ్ళినా ఘరవాలేదా? 200? 2,000?"**

ఎందుకంటే ఈ doc lo మూడు సంఖ్యలూ వస్తాయి – మరియు అవి మూడు **వేర్వేరు design లు**. "Approximate" అనే ఒక్క పదం ఆ మూడింటినీ కప్పేస్తుంది.

Part 2 – మొదటి విరుపు: రెండు అడుగుల మధ్య

3. Step – GET • check • INCR

సహజమైన code. ఇది ప్రతి tutorial lo ఉంటుంది:

```
async function allow(key) {
  const count = await redis.get(key);      // 1. చదువు
  if (count >= LIMIT) return false;        // 2. నిర్ణయించు
  await redis.incr(key);                    // 3. రాయు
  return true;
}
```

మూడు పంక్తులు, స్పష్టంగా ఉన్నాయి. ఒక్క process lo ఇది **సరైనది**.

కానీ ఆ 1 మరియు 3 మధ్య ఒక **కీటికీ** ఉంది – Redis కి వెళ్ళి రావడానికి పట్టే సమయం, రెండుసార్లు. Same-AZ lo అది సుమారు **1 ms**.

ఆ ఒక్క మిల్లీసెకనులో ఎంతమంది వస్తారు? 1,00,000 QPS దగ్గర – **100 మంది**. మరియు వాళ్ళందరూ step 1 lo **అదే పాత విలువ** ని చూస్తారు.

4. మొదటి విరుపు – పరిమితి 100, అనుమతించినవి 1,058

కొలుద్దాం. పరిమితి నిమిషానికి 100, Redis RTT 0.5 ms (కీటికీ 1 ms), వచ్చే వేగాన్ని మారుస్తూ:

పరిమితి = నిమిషానికి 100 • Redis RTT 0.5 ms (కీటికీ 1 ms)

వచ్చే వేగం	GET-then-INCR	మించినది	సూత్రం: rate × 1ms	atomic
100/s	100	+0	≈ 0	100
1,000/s	100	+0	≈ 1	100
10,000/s	107	+7	≈ 10	100
50,000/s	153	+53	≈ 50	100
1,00,000/s	189	+89	≈ 100	100

సూత్రం నిలుపు వరుసని గమనించండి - కొలత దానితో సరిపోతోంది

మించేది $\approx$ వచ్చే వేగం $\times$ కిటికీ వెడల్పు.

10,000/s $\rightarrow$ +7 (సూత్రం $\approx$ 10). 50,000/s $\rightarrow$ +53 ($\approx$ 50). 1,00,000/s $\rightarrow$ +89 ($\approx$ 100).

అంటే ఇది ఒక యాదృచ్ఛిక bug కాదు - ఇది ఒక **ఊహించదగిన భౌతిక నియమం**. మరియు అది చెప్పింది: **మీ traffic పెరిగిన కొద్దీ మీ rate limiter మరింత తప్పు అవుతుంది**. సరిగ్గా అది అవసరమైనప్పుడు.

ఇప్పుడు ఒక్కటే మార్పు - Redis ని కొంచెం **దూరంగా** పెడదాం. వేరే AZ, లేదా వేరే region:

Redis దూరంగా ఉంటే? (100,000/s, పరిమితి 100)

RTT	allowed	మించినది
0.2 ms	144	+44
0.5 ms	189	+89
1 ms	293	+193
5 ms	1058	+958

5 MS RTT - పరిమితి 100, అనుమతించినవి 1,058

మీ code ఒక్క అక్షరం మారలేదు. మీ traffic మారలేదు. **Redis ని వేరే region కి తరలించారు, అంతే.**

మరియు మీ పరిమితి ఇప్పుడు **10.6 రెట్లు**.

ఇది interview lo చెప్పడానికి చాలా బలమైన విషయం: **ఈ bug యొక్క తీవ్రత మీ code lo లేదు, మీ deployment topology lo ఉంది.**

5. Step - ఒకే ఆదేశం

పరిష్కారం సూటిగా ఉంది, మరియు అది ఒక్క వాక్యంలో ఉంది: **కిటికీని తీసేయాలి.**

చదవడం మరియు రాయడం **ఒకే ఆదేశం** అయితే, వాటి మధ్య ఎవరూ రాలేరు. Redis lo **INCR** అదే చేస్తుంది - పెంచి, కొత్త విలువ తిరిగి ఇస్తుంది:

```
// ఒకే ఆదేశం: పెంచి, కొత్త విలువ తిరిగి ఇవ్వు. ఇదే §4 యొక్క పరిష్కారం.
incrAndGet(key, ttlBucket) {
  if (!this.#up) { this.ops.failed++; throw new Error('REDIS_DOWN'); }
  this.ops.incr++;
  const k = `${key}|${ttlBucket}`;
  const v = (this.#m.get(k) || 0) + 1;
  this.#m.set(k, v);
  return v;
}
```

App వైపు:

```
const v = redis.incrAndGet(key, bucket);  
return v <= LIMIT;           // ముందే పెంచేశాం • దాటితే తిరస్కరిస్తాం
```

పట్టికలో `atomic` నిలుపు వరుస ప్రతి వరుసలోనూ **సరిగ్గా 100** – వేగం ఎంతైనా, RTT ఎంతైనా.

ఒక తలకిందుల ఆలోచన – ముందు పెంచి, తర్వాత తిరస్కరించడం

ఇక్కడ మనం limit దాటినా కూడా counter ని పెంచేస్తున్నాం. అది వృథా అనిపిస్తుంది.

కానీ అదే దీన్ని సరైనదిగా చేస్తుంది: **నిర్ణయం మరియు నమోదు ఒకే క్షణంలో జరగాలి.** "ముందు చూసి, తర్వాత పెంచుదాం" అనుకున్న క్షణంలోనే మీరు 54 కిటికీని తిరిగి తెరిచారు.

దీని ఒక దుష్ప్రభావం: counter పరిమితి కంటే చాలా ఎక్కువ పెరుగుతుంది (1,00,000 వరకు). అది సమస్య కాదు – కిటికీ ముగిసినప్పుడు అది పోతుంది. కానీ మీరు "ఎన్ని తిరస్కరించాం" అని ఆ counter నుంచి చెప్పలేరు.

Token bucket లాంటి సంక్లిష్ట algorithms కి `INCR` సరిపోదు – అక్కడ **Lua script** వాడాలి. Redis ఒక Lua script ని **అణుమాత్రంగా** నడుపుతుంది, కాబట్టి "చదువు → లెక్కించు → రాయు" మొత్తం ఒకే అడుగు అవుతుంది. ఆ algorithm యొక్క వివరాలు LLD Deep 03 §9 lo ఉన్నాయి; ఇక్కడ ముఖ్యమైనది **అది ఎక్కడ నడుస్తుంది** – మీ app lo కాదు, **Redis లోపల**.

మొదటి విరుపు పూర్తయింది. ఇప్పుడు పరిమితి ఖచ్చితంగా ఉంది. కానీ దాని ధర?

Part 3 – రెండో విరుపు: ప్రతి request కీ ఒక round-trip

6. Step – local గా లెక్కపెట్టి అప్పుడప్పుడు sync చేయడం

Atomic INCR సరైనది, కానీ అది ప్రతి ఒక్క request కీ ఒక Redis పిలుపు. 1,00,000 QPS అంటే సెకనుకి 1,00,000 Redis ఆదేశాలు, కేవలం rate limiting కోసం.

రెండు ఖర్చులు:

- **Latency** – ప్రతి request కి 0.5-1 ms జోడింపు, మీ అసలు పని మొదలవకముందే
- **Redis ఒక అడ్డంకి** – అది మీ మొత్తం traffic ని మోయాలి, మరియు అది పడితే అంతా ఆగుతుంది (§10)

సహజమైన పరిష్కారం: **local గా లెక్కపెట్టి, అప్పుడప్పుడు Redis తో sync చేయడం.** ప్రతి server తన స్వంత counter పెట్టుకుని, ప్రతి 100 ms కీ "నేను ఇన్ని వాడాను" అని చెప్తుంది.

Redis పిలుపులు 1,00,000/s నుంచి కొన్ని వందలకి పడిపోతాయి. అది పెద్ద లాభం.

దాని ఖరీదు ఎంత? కొలుద్దాం.

7. రెండో విరుపు – పరిమితి 100, అనుమతించినవి 2,000

20 servers, సెకనుకి 50,000 requests, పరిమితి 100:

20 servers · సెకనుకి 50,000 requests · పరిమితి 100

విధానం	allowed	మించినది	Redis ops/సె
ప్రతిసార్ atomic INCR	100	+0	50,000
local + 10 ms sync	487	+387	1,980
local + 50 ms sync	2000	+1900	380
local + 100 ms sync	2000	+1900	180
local + 500 ms sync	2000	+1900	20
ఒక్క server కి quota	100	+0	0

2,000 – మరియు అది ఒక పైకప్పు, ఒక యాదృచ్ఛిక సంఖ్య కాదు

50 ms sync నుంచి 500 ms వరకు అన్నీ **సరిగ్గా 2,000**. ఎందుకంటే ప్రతి server స్వతంత్రంగా 100 వరకు అనుమతిస్తుంది, మరియు servers 20.

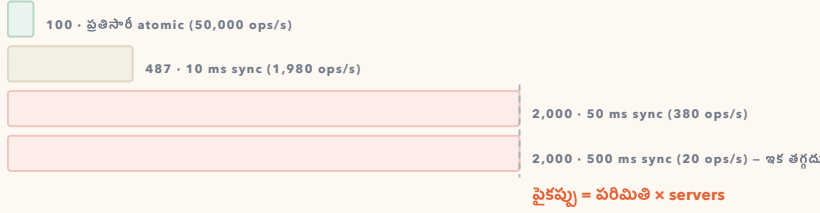
పరిమితి × servers = 100 × 20 = 2,000.

అంటే sync విరామం ఒక నిర్దిష్ట స్థాయి దాటాక, మీకు rate limiter **అస్సలు లేనట్టే** – ఒక్కో server కి ఒక్కో limiter ఉంది, అంతే.

10 ms sync కూడా 487 ఇచ్చింది – పరిమితి కంటే **4.9 రెట్లు** – మరియు దానికి ఇంకా 1,980 Redis ops/s ఖర్చు.

Sync విరామం పెరిగితే – ఖర్చు తగ్గుతుంది, తప్పు పెరుగుతుంది

అనుమతించినవి (పరిమితి 100)



Sync విరామం ఒక knob కాదు – అది ఒక **వాలు**, మరియు దాని అడుగున

"ప్రతి server కి సొంత rate limiter" ఉంది.

అక్కడ మీరు ఆదా చేసిన Redis ops కి ఇచ్చిన ధర – మీ పరిమితి.

8. Step – quota ని పంచడం • మరియు దాని దాచిన అన్యాయం

పైన చివరి వరుస చూడండి: **quota – 100 allowed, 0 Redis ops**. ఖచ్చితమైనది, మరియు ఉచితం.

ఆలోచన సులభం: sync చేయడం మానేసి, **పరిమితిని ముందే పంచేయడం**. 20 servers, పరిమితి 100 → ఒక్కో server కి **5**. ఎవరూ ఎవరితోనూ మాట్లాడనవసరం లేదు.

ఇది చాలా బాగుంది. **Traffic అన్ని servers కి సమానంగా వస్తే.**

Quota యొక్క దాచిన ఖరీదు – traffic సమానంగా లేకపోతే?

traffic ఆకారం	quota: allowed	కేంద్ర: allowed	కోల్పోయినది
సమానం (అన్ని servers)	100	100	0
కొంచెం వాలు	100	100	0
ఎక్కువ వాలు	70	100	-30
దాదాపు ఒకే server	34	100	-66

ఇప్పుడు తప్పు వ్యతిరేక దిశలో ఉంది

§7 | మనం పరిమితి కంటే ఎక్కువ అనుమతించాం. ఇక్కడ మనం తక్కువ అనుమతిస్తున్నాం – 100 కి బదులు 34.

అంటే 66% చట్టబద్ధమైన traffic ని 429 తో తిరస్కరిస్తున్నాం, మరియు మిగతా 19 servers తమ వాటాని వాడకుండా కూర్చున్నాయి.

మరియు traffic అసమానంగా ఉండటం అసాధారణం కాదు – ఒక load balancer యొక్క sticky sessions, ఒక పెద్ద customer ఒకే region నుంచి రావడం, ఒక deploy సమయంలో సగం servers మాత్రమే traffic తీసుకోవడం. ఇది మామూలే.

పరిష్కారం – వాటాని డిమాండ్ ప్రకారం పంచడం

స్థిర quota యొక్క తప్పు ఏమిటంటే అది traffic ఆకారాన్ని ముందే ఊహిస్తుంది. బదులుగా, servers తమ డిమాండ్ ని చెప్పి, దాని నిష్పత్తిలో వాటా తీసుకోనివ్వండి – అరుదుగా, కానీ న్యాయంగా.

```
// అనుకూల quota కోసం: వాడనిది వెనక్కి ఇచ్చి, డిమాండ్ చెప్పి, కొత్త వాటా తీసుకో.
lease(key, bucket, serverId, { demand, used, limit, servers }) {
  const k = `${key}|${bucket}`;
  let st = this.#m.get(k);
  if (!st || st.bucket !== bucket)
    { st = { bucket, demand: new Map(), held: new Map(), leased: 0 };
      this.#m.set(k, st); }

  // 1. ఈ server యొక్క లెక్కని సరిచేయ్ – రెండు దిశల్లోనూ.
  //   వాడనిది వెనక్కి వస్తుంది; sync కి ముందు ఆశావాదంతో వాడినది leased lo చేరుతుంది.
  const heldBefore = st.held.get(serverId) || 0;
  st.leased += (used - heldBefore);
  st.held.set(serverId, used);

  // 2. డిమాండ్ నమోదు
  st.demand.set(serverId, demand);
  let total = 0; for (const d of st.demand.values()) total += d;

  // 3. మిగిలినదాంట్లో న్యాయమైన వాటా – మరియు దాన్ని *వెంటనే* leased lo కలపడం
  const available = Math.max(0, limit - st.leased);
  const fair = total ? limit * (demand / total) : limit / servers;
  const want = Math.max(0, fair - used);
  const grant = Math.min(available, want);
  st.leased += grant;
  st.held.set(serverId, used + grant);
  return { grant, leased: st.leased };
}
```

ఈ CODE ని నేను మొదట తప్పుగా రాశాను - మరియు FUZZ 1,200 LO 705 సార్లు విఫలమైంది

నా మొదటి version lo Redis ఇలా చేసింది: `left = limit - spent` లెక్కించి, ఆ `left` lo ఒక వాటా ఇచ్చేది.

సమస్య: ఇచ్చిన వాటాని ఎక్కడా లెక్కలోకి తీసుకోలేదు. Server A కి `left` lo వాటా ఇచ్చాను. తర్వాత server B వచ్చింది - `left` ఇంకా అదే, ఎందుకంటే A దాన్ని ఇంకా ఖర్చు చేయలేదు, కేవలం తీసుకుంది.

కాబట్టి అదే మిగులుని 20 servers కీ ఇచ్చేశాను. పరిమితి 10 దగ్గర 21 అనుమతించబడ్డాయి - 2.1 రెట్లు.

పరిష్కారం ఆ ఒక్క పంక్తి: `st.leased += grant;` - ఇచ్చిన క్షణంలోనే లెక్కలో కలపాలి, ఖర్చైన క్షణంలో కాదు. అప్పుడే `available` నిజం చెప్తుంది.

ఇప్పుడు కొలుద్దాం:

20 servers · సెకనుకి 50,000 requests · పరిమితి 100 · sync 100 ms

traffic ఆకారం	కేంద్ర	స్థిర quota	అనుకూల quota	అనుకూల Redis ops/సె
సమానం	100	100	100	180
కొంచెం వాలు	100	100	100	180
ఎక్కువ వాలు	100	70	109	180
దాదాపు ఒకే server	100	34	104	180

అన్ని ఆకారాల్లోనూ 100 కి దగ్గరగా - 278 రెట్లు తక్కువ ఖర్చుతో

ఎక్కువ వాలు: స్థిర quota 70; అనుకూలం 109.

దాదాపు ఒకే server: స్థిర quota 34; అనుకూలం 104.

మరియు ఖర్చు - కేంద్రీకృత విధానం 50,000 Redis ops/s; అనుకూలం 180. 278 రెట్లు తక్కువ.

ఆ 4-9% మించడం sync ఆలస్యం నుంచి వచ్చేది, మరియు అది ఊహించదగినది - syncMs తగ్గిస్తే తగ్గుతుంది, Redis ops పెరుగుతాయి. ఇదే ఈ problem యొక్క నిజమైన knob, "approximate సరిపోతుందా" అనే ప్రశ్నకి అసలు అర్థం అదే.

రెండో విరుపు పూర్తయింది. ఖచ్చితత్వం ఉంది, ఖర్చు తక్కువ. కానీ ఇదంతా Redis ఉన్నప్పుడే.

Part 4 – మూడో విరుపు: Redis పడిపోతే

9. Step – fail open నా, fail closed నా?

Redis పడిపోయింది. మీ `allow()` function ఒక exception పట్టుకుంది. ఇప్పుడు తిరిగి ఏమి ఇవ్వాలి – `true` నా, `false` నా?

ఈ ప్రశ్నకి ఒక ప్రామాణిక జవాబు ఉంది, మరియు HLD_Design_Problems §2 కూడా దాన్నే ఇస్తుంది:

"Rate limiter down అయితే? – **Fail-open** (availability ముఖ్యం)"

తర్కం స్పష్టంగా ఉంది: మీ rate limiter ఒక **సహాయక** వ్యవస్థ. అది పడిందని మీ మొత్తం API ని ఆపేయడం తెలివితక్కువ. కాబట్టి అందరినీ వదిలేయండి.

దీన్ని కొలుద్దాం.

10. మూడో విరుపు – రెండూ తప్పు

Redis 3 సెకన్లు పడిపోయింది. సెకనుకి 50,000 requests వస్తున్నాయి, పరిమితి సెకనుకి 100:

ఆ 3 సెకన్లలో వచ్చిన requests : 1,49,956

నిజంగా అనుమతించాల్సినవి : 300

విధానం | ఆ సమయంలో అనుమతించినవి | downstream మీద భారం | ఆపేసిన నిజమైనవి

fail open (అందరినీ వదులు)	1,49,956	499.9× పరిమితి	0
fail closed (అందరినీ ఆపు)	0	0.0× పరిమితి	300
local fallback	300	1.0× పరిమితి	0

FAIL OPEN – మీ DOWNSTREAM కి పరిమితి కంటే 500 రెట్లు

మీ rate limiter ఎందుకు ఉంది? **Downstream ని కాపాడటానికి** (LLD Deep 03 §1, HLD Deep 01 §1 – అదే ఆలోచన).

Fail-open అంటే: **అది అత్యంత అవసరమైన క్షణంలో సరిగ్గా వ్యతిరేకంగా పని చేస్తుంది.** Redis పడటం తరచుగా ఒక పెద్ద సంఘటనలో భాగం – మరియు సరిగ్గా అప్పుడే మీ database కి 500 రెట్లు traffic వెళ్తుంది.

అంటే మీ rate limiter పడటం మీ **database** ని చంపుతుంది.

FAIL CLOSED – మ ఆరోగ్యకరమైన API పూర్తిగా ఆగిపోయింది

0 అనుమతి. ఆ 3 సెకన్లలో వచ్చిన **300 చట్టబద్ధమైన** requests కూడా 429 తిన్నాయి.

మీ API servers ఆరోగ్యంగా ఉన్నాయి. మీ database ఆరోగ్యంగా ఉంది. కేవలం మీ **rate limiter** యొక్క Redis పడింది – మరియు మీ మొత్తం product ఆగిపోయింది.

ఒక సహాయకవ్యవస్థ మీ ప్రధాన వ్యవస్థని కిందకి లాగింది.

రెండు ప్రామాణిక జవాబులూ చెడ్డవి. ఒకటి మీ downstream ని చంపుతుంది, ఇంకొకటి మీ product ని చంపుతుంది.

11. Step – చివరిసారి తెలిసిన వాటా

మూడో వరుస చూడండి: **local fallback – 300 అనుమతి, సరిగ్గా 1.0x పరిమితి, ఒక్క చట్టబద్ధమైన request కూడా పోలేదు.**

ఆలోచన సులభం, మరియు అది 98 నుంచే వస్తుంది: **ప్రతి server కి తన వాటా ఇప్పటికే తెలుసు.** Redis పడితే, అది "నాకు ఏమీ తెలియదు" అని కాకుండా **"నాకు చివరిసారి తెలిసిన వాటా ఇది"** అని చెప్పగలదు.

```
} catch {  
  // Redis లేదు → చివరిసారి తెలిసిన వాటాకి దిగిపో (fail open కాదు, closed కాదు)  
  this.stats.syncFailed++;  
  if (!this.#degraded) { this.#degraded = true; this.stats.fellBack++; }  
  this.#share = this.#lastGoodShare || this.limit / this.#servers;  
}
```

FAIL OPEN కాదు • FAIL CLOSED కాదు • FAIL DEGRADED

ఇది మూడో దారి, మరియు దీనికి ఒక పేరు ఉండాలి: **ఆఖరి మంచి స్థితికి దిగిపోవడం.**

Redis ఉన్నప్పుడు: డిమాండ్ ప్రకారం న్యాయమైన పంపకం, 4-9% లోపంతో.

Redis లేనప్పుడు: సమాన పంపకం, ఖచ్చితమైన మొత్తం పరిమితితో.

మీరు కోల్పోయేది న్యాయం, పరిమితి కాదు. అది సరైన రాజ్ – ఎందుకంటే outage సమయంలో "ఏ server ఎంత వాడింది" అనేది "downstream బతికిందా" అనే దాని కంటే తక్కువ ముఖ్యం.

కానీ – రెండు దాచిన పగుళ్ళు ఉన్నాయి

ఈ fallback ని నేను రాశాక, fuzz ఇంకా **20 సార్లు విఫలమైంది.** కారణాలు రెండు, మరియు రెండూ ఒకే రకమైనవి.

ఒకటి – రెండు వేర్వేరు budgets.

కిటికీ 0: అనుమతి 416 • పరిమితి 208 • Redis down: true

సరిగ్గా **2 × 208**. కారణం: Redis ఉన్నంతవరకు అది తన counter తో 208 వరకు అనుమతించింది. తర్వాత అది పడింది, మరియు fallback **సున్నా నుంచి** తన సొంత 208 మొదలుపెట్టింది.

రెండు లెక్కలు, ఒకే కిటికీ. పరిష్కారం: **Redis దారిలో కూడా local లెక్క పెట్టాలి**, తద్వారా fallback సరైన చోట నుంచి కొనసాగుతుంది.

```
// ముఖ్యం: Redis దారిలో కూడా local లెక్క పెట్టాలి.  
// లేకపోతే Redis పడినప్పుడు fallback సున్నా నుంచి మొదలై,  
// *రెండో* పూర్తి పరిమితిని ఇచ్చేస్తుంది - §10 చూడండి.  
if (ok) { this.#used++; this.#committed++; this.stats.allowed++; }  
else this.stats.blocked++;
```

రెండు – Redis తిరిగి వచ్చాక అది ఏమే తెలియనట్టు కొనసాగుతుంది.

అది సరిచేశాక కూడా 303 వచ్చింది. కారణం: outage సమయంలో local గా అనుమతించిన వాటిని **Redis కి ఎవరూ చెప్పలేదు**. అది తిరిగి లేచి, తన **పాత** లెక్క నుంచి కొనసాగి, ఇంకొక budget ఇచ్చింది.

```
// Redis తిరిగి వచ్చిందా? అయితే outage lo local గా ఇచ్చినవి ముందు చెప్పాలి.  
// లేకపోతే Redis తన పాత లెక్క నుంచి కొనసాగి రెండోసారి పరిమితి ఇస్తుంది.  
if (this.#used > this.#committed) {  
    this.#redis.incrBy(key, b, this.#used - this.#committed);  
    this.#committed = this.#used;  
}
```

ఈ రెండు BUGS ఒకే ఆకారంలో ఉన్నాయి

రెండూ ఒకే తప్పు: **ఒక కిటికీకి రెండు లెక్కలు.**

మొదటిది – Redis ఒకటి, fallback ఒకటి. రెండోది – outage కి ముందు ఒకటి, తర్వాత ఒకటి.

నియమం ఒక్క వాక్యంలో: **ఒక కిటికీకి ఒకే budget. అది ఎక్కడ నిల్వ ఉన్నా, ఎవరు ఖర్చు చేసినా.**

మరియు ఈ రెండింటినీ కనిపెట్టింది ఒకే నియమం – §13 నియమం 3. Unit test ఏదో ఈ ఆకారాన్ని పట్టుకునేది కాదు, ఎందుకంటే ఇది **outage సరిగ్గా కిటికీ మధ్యలో** వచ్చినప్పుడే కనిపిస్తుంది.

Part 5 – పూర్తి system

12. Step – ఒక bucket, ఒక budget

§11 చివర్లో వచ్చిన నియమాన్ని code lo ఎలా అమలు చేయాలి?

ప్రతి లెక్కా ఒక bucket కి కట్టబడి ఉండాలి, మరియు bucket అనేది సమయం నుంచి వచ్చే సంఖ్య – ఎవరైనా, ఎప్పుడైనా అదే విలువ లెక్కించగలరు:

```
#bucketOf(t) { return Math.floor(t / this.#windowMs); }

allow(key = 'default') {
  const now = this.#clock(), b = this.#bucketOf(now);
  if (b !== this.#bucket) { // కొత్త కిటికీ
    this.#bucket = b; this.#demand = 0; this.#used = 0; this.#committed = 0;
    this.#share = this.#lastGoodShare || this.limit / this.#servers;
    this.#nextSync = now;
  }
  this.#demand++;
  // ఇక్కడి నుంచి: central అయితే §5 యొక్క incrAndGet,
  // adaptive అయితే §8 యొక్క lease – రెండూ పైన పూర్తిగా ఉన్నాయి.
}
```

ఇది HLD Deep 01 §12 lo చూసిన అదే ఆలోచన: **సమయం నుంచి వచ్చే key, యాదృచ్ఛికత నుంచి కాదు.** అక్కడ అది `scheduledFor`; ఇక్కడ అది `bucket`. రెండింటిలోనూ కారణం ఒకటే – **ఏ server అయినా, restart తర్వాత అయినా, అదే జవాబు రావాలి.**

Redis వైపు కూడా ప్రతి record bucket ని పట్టుకుని ఉంటుంది, మరియు bucket మారితే అది **తనని తాను** శుభ్రం చేసుకుంటుంది:

```
if (!st || st.bucket !== bucket)
  { st = { bucket, demand: new Map(), held: new Map(), leased: 0 };
    this.#m.set(k, st); }
```

13. మొత్తం code · 48 లక్షల requests · mutation testing

ఐదు నియమాలు

#	నియమం	విరిగితే అర్థం
1	ప్రతి request అనుమతి లేదా తిరస్కరణ – రెండూ కాదు, ఏదీ కాదు అనేది లేదు	<code>allow()</code> ఒక దారిలో ఏమీ తిరిగి ఇవ్వడం లేదు
2	Redis ఆర్గ్యంగా ఉన్న కిటికీలో పరిమితి దాటకూడదు	atomicity విరిగింది (§4)
3	Redis పడిన కిటికీలో కూడా పరిమితి × servers దాటకూడదు	fail-open అయింది (§10)
4	<code>allow()</code> ఎప్పుడూ exception వినరకూడదు	Redis లోపం బయటికి పోతోంది
5	లెక్కలు ఎప్పుడూ ఋణాత్మకం కావు	lease వెనక్కి ఇవ్వడంలో తప్పు

నియమం 2 lo అనుకూల quota కి కొంత **సహనం** ఇచ్చాను – servers సంఖ్యకి సమానంగా. ఎందుకంటే §8 lo చూసినట్టు sync ఆలస్యం వల్ల కొంత మించడం **నిర్మాణపరమైనది**, ఒక bug కాదు. కేంద్రీకృత విధానానికి సహనం **సున్నా**.

ప్రతి ప్రయోగంలో 1-20 servers, యాదృచ్ఛిక పరిమితి, కిటికీ, sync విరామం, విధానం, మరియు 40% ప్రయోగాల్లో ఒక యాదృచ్ఛిక Redis outage:

1,200 యాదృచ్ఛిక ప్రయోగాలు · 48,00,000 requests
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

allowed 14,81,780
blocked 33,18,220
fellBack 1,04,626
syncs 7,38,600
syncFailed 38,455

Test విఫలం కాగలదా? – ఐదు మార్పులు

మార్పు లేని code → 0/1200 విఫలం

atomic కాకుండా GET-then-INCR చేస్తే → 148/400 విఫలం
ఉదా: కిటికీ 0: 18 అనుమతి · పరిమితి 17 (+0 సహనం)

Redis పడితే fail-open చేస్తే → 17/400 విఫలం
ఉదా: కిటికీ 1 (Redis down): 428 అనుమతి – fail-open అయింది

lease ఇచ్చినదాన్ని leased lo కలపకపోతే → 227/400 విఫలం
ఉదా: కిటికీ 0: 31 అనుమతి · పరిమితి 10 (+13 సహనం)

outage తర్వాత Redis కి చెప్పకపోతే → 5/400 విఫలం
ఉదా: కిటికీ 0 (Redis down): 303 అనుమతి – fail-open అయింది

Redis దారిలో local లెక్క పెట్టకపోతే → 5/400 విఫలం
ఉదా: కిటికీ 0 (Redis down): 321 అనుమతి – fail-open అయింది

ఐదూ పట్టుబడ్డాయి. కానీ చివరి రెండింటి సంఖ్యలు చూడండి – **400 lo కేవలం 5.**

5/400 అంటే ఆ నియమం బలహీనమైనదని కాదు - ఆ BUG అరుదైనదని

ఆ రెండు bugs కనిపించాలంటే మూడు విషయాలు **ఒకసారి** జరగాలి: (1) Redis పడాల్సి, (2) అది ఒక కిటికీ మధ్యలో పడాల్సి, (3) ఆ కిటికీలో అప్పటికే గణనీయమైన traffic వాడి ఉండాలి.

నా ప్రయోగాల్లో 40% lo outage ఉంది, మరియు వాటిలో కొన్నింటిలోనే ఈ కలయిక వస్తుంది. **1.25% - అదే ఈ bug యొక్క నిజమైన తరచుదనం.**

మరియు అదే దాన్ని ప్రమాదకరంగా చేస్తుంది: **ఇది మీ staging lo ఎప్పటికీ కనిపించదు.** ఇది మీకు ఒకసారి, production lo, ఒక పెద్ద outage మధ్యలో కనిపిస్తుంది - సరిగ్గా మీరు దాన్ని debug చేయలేని క్షణంలో.

యాదృచ్ఛిక ప్రయోగాలు ఇలాంటివాటికే. ఒక unit test ఈ మూడు పరిస్థితులనూ కలిపి రాయాలని ఎవరికీ తోచదు.

దశల నుంచి ఇక్కడికి - ఏమి చేరింది

ఎక్కడ	ఏమి జోడించాం	ఏమి బాగుపడింది
§3	<code>GET</code> + check + <code>INCR</code>	పని చేస్తుంది · తక్కువ traffic దగ్గర
§5	<code>incrAndGet</code> (ఒకే ఆదేశం)	1,058 → సరిగ్గా 100 , RTT ఎంతైనా
§8	<code>lease()</code> అనుకూల quota	Redis ops 50,000 → 180 , లోపం 4-9%
§8	<code>st.leased += grant</code>	2.1× మించడం → 0%
§11	<code>lastGoodShare</code> fallback	Redis outage lo 500x → 1.0x
§11	Redis దారిలో local లెక్క	రెండు budgets → ఒకటి
§11	<code>incrBy</code> reconciliation	outage తర్వాత రెండే budget లేదు
§12	<code>bucket</code> ప్రతి లెక్కకీ	కిటికీ మారితే అంతా తనని తాను శుభ్రం చేసుకుంటుంది

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-4	"Algorithm ఎంపిక సులభమైన భాగం" అని చెప్పి, చదువు→లెక్చరు→రాయి బొమ్మ గీయండి
4-8	§2 ప్రశ్నలు. ముఖ్యంగా "110 ఫరవాలేదా? 200? 2,000?"
8-14	GET / INCR రాసి, overshoot $\approx$ rate $\times$ window సూత్రం ఇవ్వండి. 1,058 సంఖ్య చెప్పండి
14-20	INCR /Lua. "నిర్ణయం మరియు నమోదు ఒకే క్షణంలో"
20-30	రెండో విరుపు. local sync → పరిమితి $\times$ servers. తర్వాత quota → 66% అన్యాయం. తర్వాత అనుకూల quota
30-38	మూడో విరుపు. fail-open = 500x, fail-closed = మీ product down. మూడో దారి
38-42	ఒకే bucket ఒకే budget · రెండు budgets bug
42-45	ఐదు నియమాలు, మరియు "ఆ bug 400 lo 5 సార్లే కనిపించింది"

ఏమి తప్పక చెప్పాలి

1. **Overshoot $\approx$ rate $\times$ window.** ఇది ఒక సూత్రం, ఒక అభిప్రాయం కాదు – మరియు అది మీ Redis ఎంత దూరంలో ఉందో దాని మీద ఆధారపడుతుంది.
2. **నిర్ణయం మరియు నమోదు ఒకే అడుగు కావాలి.** INCR , లేదా Lua.
3. **Local counting తో ఫైకప్పు = పరిమితి $\times$ servers.** Sync విరామం ఒక knob కాదు, ఒక వాలు.
4. **స్థిర quota traffic సమానంగా ఉంటే న్యాయం.** వాలు ఉంటే 66% చట్టబద్ధమైన traffic పోతుంది.
5. **Fail-open మీ downstream ని చంపుతుంది; fail-closed మీ product ని చంపుతుంది.** మూడో దారి – చివరిసారి తెలిసిన వాటా.

ఏమి వదిలేయాలి

- ఐదు algorithms యొక్క వివరాలు – "token bucket, Lua script lo" అని ఒక వాక్యం చాలు (LLD Deep 03 lo పూర్తిగా ఉంది)
- Redis cluster topology – అడిగితే మాత్రమే
- 429 response యొక్క headers (Retry-After , X-RateLimit-*) – ఒక వాక్యం
- Sliding window యొక్క సరిహద్దు గణితం – ఇది HLD ప్రశ్న, LLD ప్రశ్న కాదు

15. నీటితో చెప్పాల్సిన English script

"Every rate limiting algorithm – token bucket, sliding window, all of them – has the same shape: **read, compute, write**. In one process that's a single step. Distributed, it's three separate moments with the whole world moving in between. So I'd say the algorithm choice is the easy part here; everything hard comes from **where the counter lives**."

"The natural first version is `GET`, compare, `INCR`. There's a window between the read and the write – two round trips, about a millisecond same-AZ – and **everyone who arrives in that window sees the same stale value**. I measured it, and it follows a formula: **overshoot $\approx$ rate $\times$ window**. At 10k/s it's +7, at 50k it's +53, at 100k it's +89. The predictions and the measurements line up, so this isn't a flaky bug – it's physics. And it means **the limiter gets more wrong as traffic grows**, which is exactly when you need it."

"Then I moved Redis further away – not a code change, a topology change. At 5ms RTT, a limit of 100 lets through **1,058**. Ten times over, from a deployment decision."

"Fix is to collapse the window: make the decision and the record the same step. `INCR` returns the new value, so I increment first and reject if I'm over. That looks wasteful – I'm incrementing past the limit – but that's what makes it correct. The moment you think 'let me check first, then increment', you've reopened the window. For token bucket you can't do it in one command, so it goes in a **Lua script**, which Redis runs atomically. The point isn't the algorithm, it's that **it runs inside Redis, not in your app**."

"That's exact, but it's a Redis call on every single request – 100k ops/s just for rate limiting, plus latency on every request, plus Redis as a bottleneck. So the obvious optimization is local counting with periodic sync. I measured that too, and it's worse than people expect: with 20 servers, anything past a 50ms sync interval allows exactly **2,000** against a limit of 100. That's not noise, it's a ceiling – **limit $\times$ servers**, because each server independently allows the full limit. Past a point you don't have a distributed rate limiter, you have twenty separate ones."

"Splitting the quota – five each for twenty servers – is exact and free, zero Redis calls. But it assumes even traffic. Under skew I measured it allowing **34 instead of 100**: it rejects 66% of legitimate traffic while nineteen servers sit on unused quota. So instead servers report **demand** and lease a proportional share. That lands at 104–109 against a limit of 100 under heavy skew, using **180 Redis ops per second instead of 50,000** – 278× cheaper. The 4–9% error comes from sync lag and it's tunable; that's the real meaning of 'is approximate okay'."

"Last, Redis dies. The standard answer is fail-open, and I'd push back on it with a number. During a three-second outage at 50k/s, fail-open let through **149,956 requests against a budget of 300 – 500× the limit** – straight at the downstream the limiter exists to protect. Fail-closed allowed zero, so a healthy API went down because an auxiliary system's Redis went down. **Both standard answers are wrong**. The third option is that each server already knows its share, so it degrades to that: I measured exactly 300 – 1.0× the limit, zero legitimate requests lost. You lose fairness, not the limit."

"Two bugs the fuzz found there, and they're the same bug twice. First, a window had **two budgets** – Redis allowed its full limit, then the fallback started a fresh one from zero, giving exactly 2×. Second, after recovery Redis resumed from its stale count because nobody told it what the fallback had allowed. The rule is **one budget per window, wherever it's stored and whoever spends it**. Those only showed up in 5 of 400 runs, because they need an outage landing mid-window with traffic already spent – which is precisely why they'd never appear in staging."

16. Follow-ups – hot keys, బహుళ ప్రాంతాలు, tier-based limits

"ఒక key చాలా hot అయితే – ఒక పెద్ద customer, లేదా ఒక దాడి?"

Redis cluster lo ఒక key **ఒకే shard** మీద ఉంటుంది. కాబట్టి ఒక hot key అంటే ఒక hot shard – మిగతా shards ఖాళీగా ఉండగా ఒకటి మునిగిపోతుంది. పరిష్కారం **key splitting**: `user:123` కి బదులు `user:123:{0..15}` అనే 16 keys, ఒక్కొక్కటి పరిమితిలో 1/16 వంతుతో. ఇది §8 యొక్క quota ఆలోచన, కానీ servers మధ్య కాకుండా **shards మధ్య**. మరియు అదే బలహీనత వర్తిస్తుంది – traffic ఆ 16 lo సమానంగా పడాలి, కాబట్టి request ని hash చేసి పంచాలి, యాదృచ్ఛికంగా కాదు.

"Users ప్రపంచమంతా ఉంటే – ప్రతి region కీ ఒక Redis నా?"

§4 lo చూసినదాన్ని గుర్తుంచుకోండి: cross-region RTT (~50–150 ms) తో కేంద్రీకృత లెక్క పూర్తిగా అర్థరహితం – overshoot వేల కొద్దీ ఉంటుంది. కాబట్టి ఒక్కో region కి సొంత Redis, మరియు పరిమితిని **regions మధ్య**

పంచడం – సరిగ్గా §8. Regions మధ్య అనుకూల పంపకం చేయాలంటే sync విరామం పెద్దది (సెకన్లు), కాబట్టి లోపం ఎక్కువ. **ఇది అంగీకరించాల్సిన నిజం:** ప్రపంచవ్యాప్త ఖచ్చితమైన rate limit అనేది ఉండదు.

"ఒక్క customer కి వేరే పరిమితి (free / pro / enterprise)?"

పరిమితి ఒక **విలువ**, ఒక స్థిరాంకం కాదు – `allow(key)` కి ముందు ఆ key యొక్క tier చూసి పరిమితి తెచ్చుకోవాలి. ఆ lookup ని cache చెయ్యండి (tier అరుదుగా మారుతుంది), లేకపోతే మీరు §6 lo తప్పించుకున్న round-trip ని తిరిగి కొనుక్కున్నట్లే. మరియు ఒక ఆచరణాత్మక విషయం: **పరిమితి మారినప్పుడు నడుస్తున్న కిటికీ ఏమవుతుంది?** సులభమైన జవాబు – కొత్త పరిమితి తర్వాతి కిటికీ నుంచి వర్తిస్తుంది.

"429 తో ఏమి పంపాలి?"

`Retry-After` తప్పనిసరి – క్లయింట్ ఎప్పుడు మళ్ళీ ప్రయత్నించాలో తెలియకపోతే అది **వెంటనే** ప్రయత్నిస్తుంది, మరియు మీరు ఒక retry storm కొనుక్కుంటారు (LLD Deep 25 §4 lo దాన్ని కొలిచాం – jitter లేని backoff **ఏమీ ఆదా చేయలేదు**). `X-RateLimit-Remaining` ఇవ్వడం ఉపయోగకరం, కానీ §8 తర్వాత అది ఒక **అంచనా** మాత్రమే – మీ server కి తన వాటా మాత్రమే తెలుసు.

"Rate limiter ని API gateway lo పెట్టాలా, ప్రతి service lo పెట్టాలా?"

Gateway lo పెడితే ఒక్క చోట, కానీ అది ఒక SPOF మరియు service-specific పరిమితులు కష్టం. Service lo పెడితే సౌకర్యం, కానీ ప్రతి service కీ ఇదే code. ఆచరణలో **రెండు** – gateway lo ఒక ఉదారమైన global limit (DDoS కోసం), service lo ఒక ఖచ్చితమైన business limit.

17. ఏమి నేర్చుకున్నాం

1. Distributed అంటే algorithm మారడం కాదు, అడుగుల మధ్య దూరం రావడం. "చదువు → లెక్కించు → రాయు" ఒక్క process lo ఒక అడుగు; 50 servers తో మూడు క్షణాలు. **ఈ doc lo ఒక్క algorithm కూడా లేదు, అయినా మూడు విరుపులు ఉన్నాయి.**

2. Overshoot $\approx$ rate $\times$ window. ఇది ఒక సూత్రం, మరియు కొలత దానితో సరిపోయింది. దాని అర్థం – **traffic పెరిగిన కొద్దీ rate limiter మరింత తప్పు అవుతుంది**, మరియు Redis దూరం పెరిగిన కొద్దీ కూడా. 0.5 ms $\rightarrow$ 5 ms అంటే +89 $\rightarrow$ **+958**.

3. నిర్ణయం మరియు నమోదు ఒకే అడుగు కావాలి. పరిమితి దాటినా counter పెంచడం వృథా అనిపిస్తుంది, కానీ అదే దాన్ని సరైనదిగా చేస్తుంది.

4. Local counting యొక్క ఫైకప్పు = పరిమితి $\times$ servers. Sync విరామం ఒక knob కాదు – అది ఒక వాలు, మరియు దాని అడుగున rate limiter అనేదే ఉండదు.

5. ఖచ్చితత్వం మరియు ఖర్చు మధ్య నిజమైన knob `syncMs`. కేంద్రీకృతం 50,000 ops/s తో 0% లోపం; అనుకూలం **180 ops/s తో 4-9%**. "Approximate సరిపోతుందా" అనే ప్రశ్న అడిగేది సరిగ్గా ఇదే.

6. రెండు ప్రామాణిక జవాబులూ తప్పు కావచ్చు. Fail-open **500x** భారం; fail-closed మీ product down. మూడో దారి ఉందని తెలియాలంటే ముందు **రెండింటినీ కొలవాలి.**

7. "ఇచ్చాను" అనేదాన్ని వెంటనే లెక్కలో కలపాలి, "ఖర్చైంది" అన్నప్పుడు కాదు. నా lease code `limit - spent` నుంచి వాటా ఇచ్చేది, కానీ ఇచ్చినదాన్ని ఎక్కడా తీసివేయలేదు – కాబట్టి అదే మిగులు 20 servers కీ వెళ్ళింది. `st.leased += grant` – ఒక్క పంక్తి, 2.1x తేడా.

8. ఒక కీటికీ ఒక budget. Redis ఒకటి + fallback ఒకటి = 2x. Outage కి ముందు ఒకటి + తర్వాత ఒకటి = ఇంకో budget. రెండూ ఒకే తప్పు, వేరే వేషాల్లో.

9. అరుదైన bug అంటే చిన్న bug కాదు. ఆ చివరి రెండు bugs 400 ప్రయోగాల్లో 5 సార్లే కనిపించాయి – ఎందుకంటే వాటికి outage కీటికీ మధ్యలో రావాలి. అంటే అవి మీ staging lo ఎప్పటికీ కనిపించవు, మరియు production lo ఒక పెద్ద outage మధ్యలో మాత్రమే కనిపిస్తాయి.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Rate limiter యొక్క నిజమైన ప్రశ్న "ఎంత అనుమతించాలి" కాదు – "ఎన్ని అనుమతించామో ఎవరికి తెలుసు" అనేది.

§4 lo ఆ జ్ఞానం పాతది (GET మరియు INCR మధ్య). §7 lo అది చెల్లాచెదురు (ప్రతి server కి సొంత సత్యం). §10 lo అది లేదు (Redis పడింది).

మూడు పరిష్కారాలూ ఒకే పని చేస్తాయి – ఆ జ్ఞానాన్ని ఒకే చోటికి, ఒకే క్షణానికి, ఒకే budget కి కట్టడం.